

Universidad de San Carlos de Guatemala

Centro Universitario de Occidente CUNOC

División de las Ciencias de la Ingeniería

Organización de Lenguajes y Compiladores 2

Sección: A

Docente: Mario Moisés Ramírez Tobar



“Manual de Usuario Proyecto 1, Fase 1”

Estudiante:

Mynor Miguel Monzón Martínez 202230884

Quetzaltenango, 26 de septiembre de

2026

Introducción

Este manual describe el uso del compilador del Proyecto 1 “Contacto”, una aplicación de escritorio desarrollada en Java que permite escribir, editar y compilar programas en tres lenguajes distintos desde una sola interfaz gráfica: Y?, Zetariano y Pig Latin.

Los tres lenguajes se diseñaron con fines educativos para el curso de Organización de Lenguajes y Compiladores 2, y trabajan juntos: Y? define estructuras y funciones libres con una sintaxis parecida a Python (bloques por indentación); Zetariano define clases con atributos y métodos, con una sintaxis parecida a Java; y Pig Latin —de sintaxis inspirada en latín— es el lenguaje principal de cada programa, el que se ejecuta, y que `IMPORTA` las clases de Zetariano y las estructuras/funciones de Y? que necesite.

A diferencia de un simple analizador, esta aplicación **SÍ** produce un programa que se puede ejecutar de verdad: al compilar, además de revisar que el código esté bien escrito, genera un archivo de código en lenguaje C, listo para compilarse con un compilador de C (como `gcc`) y correr como cualquier otro programa. Ese último paso —compilar el `.c` con `gcc` y ejecutar el binario resultante— se hace fuera de la aplicación, en una terminal; la sección 4.11 explica cómo.

1. Requisitos del Sistema

Para usar el compilador se necesita lo siguiente:

- Sistema operativo: Windows, macOS o Linux.
- Java Development Kit (JDK) versión 21 o superior.
- Apache Maven, para compilar el proyecto desde el código fuente.
- Un compilador de C compatible con el estándar C11 (por ejemplo `gcc`), instalado y disponible en la terminal, para compilar el código C que genera la aplicación y obtener un programa ejecutable. En Ubuntu se instala con: `sudo apt install gcc`

2. Instalación y Ejecución

2.1 Obtener el programa

El proyecto se obtiene clonando el repositorio de GitHub del curso, o descargando el código fuente como archivo comprimido desde ahí mismo.

2.2 Compilar el proyecto

Desde la carpeta raíz del proyecto (donde está el archivo `pom.xml`), ejecutar en una terminal:

```
mvn clean compile
```

Este comando descarga las dependencias, genera los analizadores a partir de las tres gramáticas y compila todo el código fuente. Es obligatorio usar `clean` si se modificó alguna gramática; de lo contrario `mvn compile` es suficiente.

2.3 Ejecutar la aplicación

Para abrir la interfaz gráfica:

```
java -cp target/classes contacto.comun.ui.VentanaPrincipal
```

También se le puede indicar una carpeta de proyecto para que se abra directamente con ella cargada en el árbol de archivos:

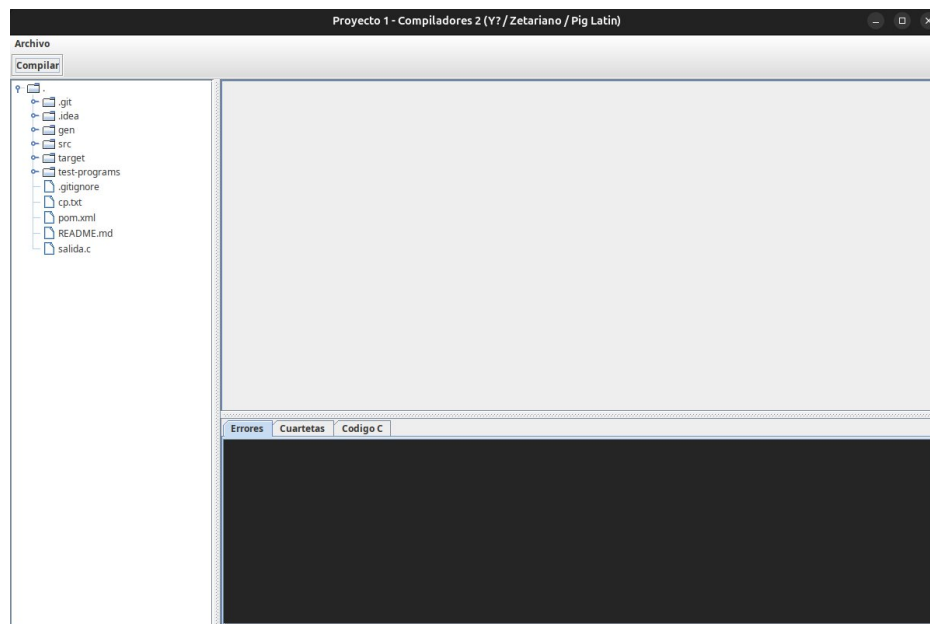
```
java -cp target/classes contacto.comun.ui.VentanaPrincipal ruta/a/mi/proyecto
```

Ambos comandos también se pueden ejecutar directamente desde IntelliJ IDEA o NetBeans, corriendo la clase VentanaPrincipal como programa principal.

3. Interfaz Principal

Al iniciar la aplicación se abre una única ventana dividida en las siguientes áreas:

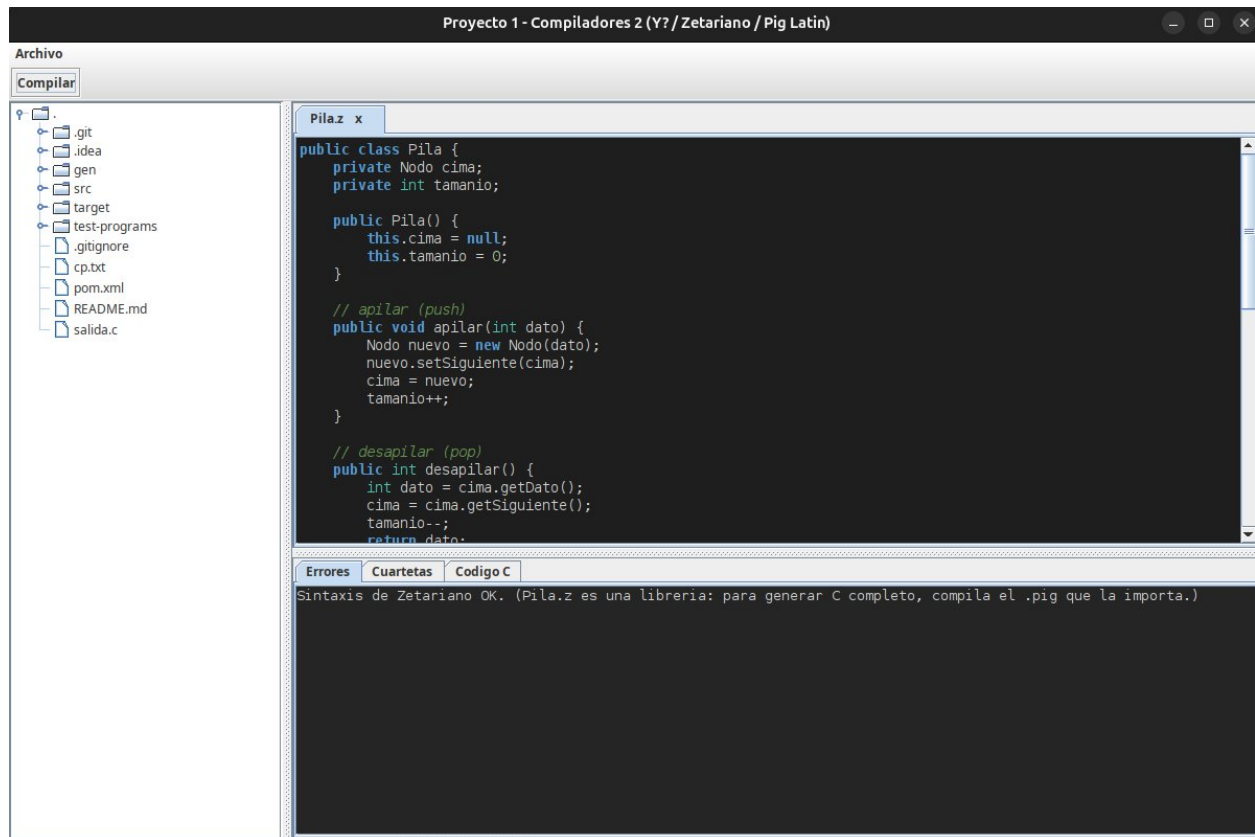
Área	Descripción
Árbol de archivos	Columna a la izquierda. Muestra la carpeta de proyecto abierta, con sus subcarpetas y archivos. Las carpetas se muestran primero, en orden alfabético.
Editor de código	Área central, con una pestaña por cada archivo abierto. Utiliza fuente monoespaciada y aplica coloreado de sintaxis en tiempo real, distinto para cada uno de los tres lenguajes.
Consola de salida	Panel inferior con tres pestañas: Errores, Cuartetas y Código C. Muestra el resultado de la última compilación.
Barra de herramientas	Franja superior con el botón Compilar.
Barra de menú	Contiene el menú Archivo.



3.1 Coloreado de sintaxis

El editor colorea el código automáticamente conforme se escribe, sin necesidad de compilar. El coloreado es específico para cada uno de los tres lenguajes (la aplicación lo determina por la extensión del archivo: .y, .z o .pig) y usa los siguientes colores:

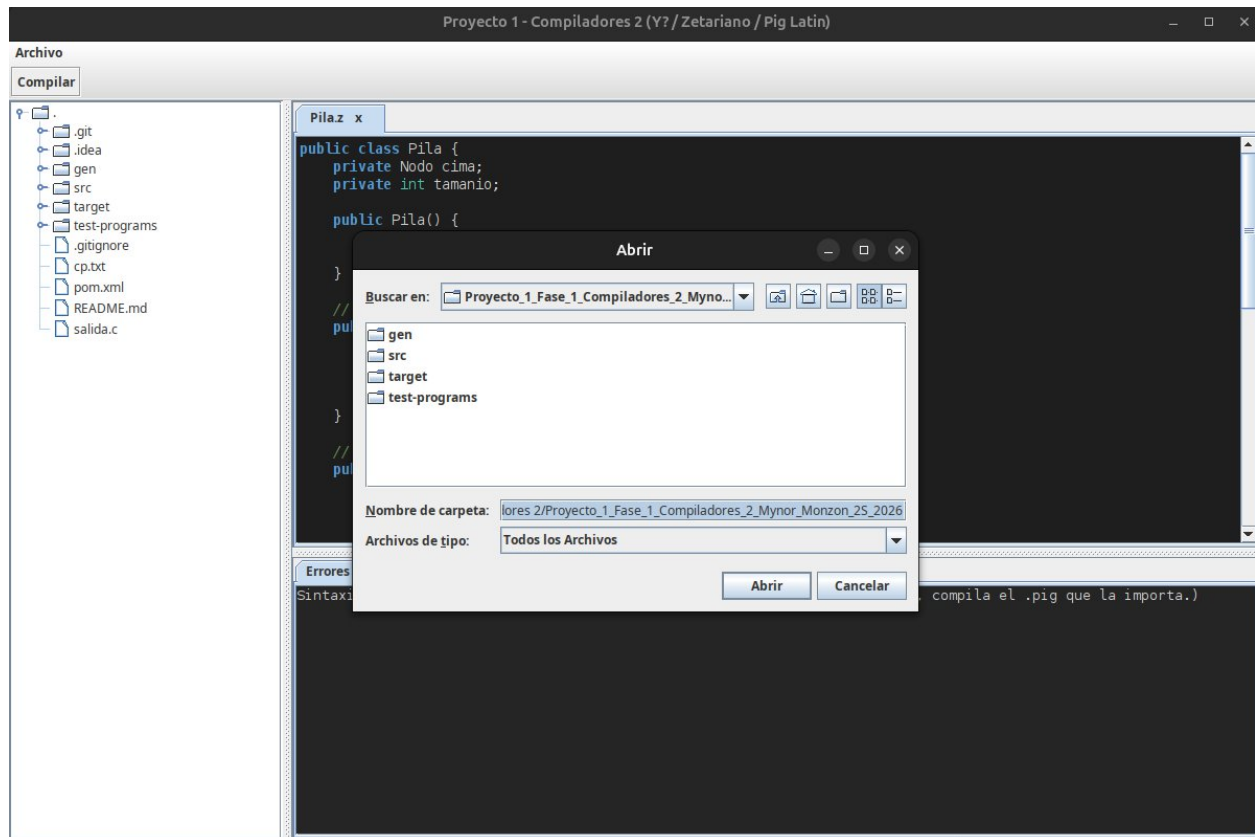
Elemento	Color
Palabras reservadas	Azul, negrita
Tipos de datos	Verde agua, negrita
Cadenas y caracteres	Naranja
Números	Verde claro
Comentarios	Verde oscuro, cursiva
Identificadores y el resto	Color de texto normal



4. Funcionalidades del Sistema

4.1 Abrir una carpeta de proyecto

Ir al menú Archivo y seleccionar Abrir carpeta. Se muestra un selector de carpetas; al elegir una, el árbol de archivos se actualiza para mostrar su contenido.



4.2 Crear un archivo nuevo

Hacer clic derecho sobre una carpeta en el árbol de archivos y seleccionar Nuevo archivo. Se pide el nombre, que debe incluir la extensión correspondiente (.y, .z o .pig). El archivo se crea vacío; para editarlo hay que abrirlo con doble clic.

4.3 Crear una carpeta nueva

Igual que el punto anterior, pero seleccionando Nueva carpeta en el menú contextual.

4.4 Abrir un archivo

Hacer doble clic sobre el archivo en el árbol. Se abre en una pestaña nueva del editor, ya coloreado. Si el archivo ya estaba abierto, la aplicación cambia a esa pestaña en vez de abrirlo dos veces.

4.5 Guardar un archivo

Ir al menú Archivo y seleccionar Guardar, o presionar Control + S. Si el archivo ya tiene una ruta asociada, se guarda directamente sobre ella.

4.6 Guardar como / Descargar una copia

Ir al menú Archivo y seleccionar Guardar como (la opción Descargar hace exactamente lo mismo, pensada para cuando lo que se quiere es sacar una copia del archivo hacia otra carpeta). Siempre solicita una ubicación nueva.

4.7 Cerrar una pestaña

Cada pestaña tiene un pequeño botón “x” para cerrarla. Si el archivo tiene cambios sin guardar, la pestaña muestra un asterisco (*) junto al nombre, y al intentar cerrarla aparece un aviso preguntando si se quiere cerrar de todos modos sin guardar.

4.8 Renombrar o eliminar un archivo o carpeta

Clic derecho sobre el archivo o carpeta en el árbol y seleccionar Renombrar o Eliminar. Eliminar pide confirmación antes de borrar, porque la acción no se puede deshacer.

4.9 Compilar un programa

Con un archivo .pig abierto y con el foco en su pestaña, presionar el botón Compilar de la barra de herramientas. Antes de compilar, la aplicación guarda automáticamente TODOS los archivos abiertos que tengan cambios sin guardar, no solo el que está activo —así que si se compila justo después de editar una clase que el .pig importa, la compilación siempre usa la versión más reciente, aunque no se haya guardado a mano.

Solo se puede compilar un archivo .pig completo (con todo lo que importa). Si se presiona Compilar con un .z o un .y abierto en la pestaña activa, la aplicación solo revisa que ese archivo esté bien escrito (léxica y sintácticamente), porque un archivo de ese tipo por sí solo no forma un programa completo —está pensado para que un .pig lo importe.

4.10 Revisar los resultados de la compilación

El resultado aparece en la consola de salida, repartido en tres pestañas:

Pestaña	Contenido
Errores	Si la compilación fue exitosa, el mensaje “Compilación OK, sin errores”. Si hubo errores, la lista completa, cada uno con el archivo, la línea, la columna y una descripción.
Cuartetas	El código de tres direcciones (cuartetas) generado para el .pig y para cada archivo que importa, en el orden en que se compilaron.
Código C	El código en lenguaje C generado a partir de las cuartetas, listo para compilarse con gcc. Este mismo código se guarda automáticamente en un archivo junto al .pig (ver 4.11).

[Captura de pantalla: las tres pestanas de la consola de salida despues de compilar main.pig sin errores]

[Captura de pantalla: la pestana Errores mostrando una lista de errores, cada uno con archivo, linea y columna]

4.11 Ejecutar el programa generado

Compilar un .pig sin errores genera, además de lo que se muestra en pantalla, un archivo con el mismo nombre y extensión .c en la misma carpeta del .pig (por ejemplo, compilar main.pig genera main.c junto a él). Ese archivo se actualiza cada vez que se vuelve a compilar.

Para obtener un programa ejecutable a partir de ese archivo, hay que compilarlo con un compilador de C desde una terminal, fuera de la aplicación:

```
gcc -std=c11 main.c -o main
./main
```

(En Windows, el ejecutable resultante se llama main.exe y se corre escribiendo main o .\main desde la terminal, según la consola que se use.) A partir de ahí el programa funciona como cualquier otro ejecutable: si el programa fuente usa las instrucciones de lectura del lenguaje, va a pedir datos por teclado; si usa las de impresión, va a mostrar texto en la terminal.

5. Referencia de los Tres Lenguajes

Esta sección resume la sintaxis de cada uno de los tres lenguajes. Los tres comparten los mismos cinco tipos de datos con la misma jerarquía de conversión (de mayor a menor nivel: cadena/textum, decimal/decimalis, entero/numerus, carácter/littera, booleano/bool), aunque cada uno los llama distinto.

5.1 Lenguaje Y?

Y? es un lenguaje imperativo cuyos bloques se delimitan por INDENTACIÓN (como Python), no por llaves. Es sensible a mayúsculas y minúsculas. Su extensión de archivo es .y.

5.1.1 Estructura de un programa

Un archivo .y tiene una sección de estructuras (opcional) y una sección de funciones (obligatoria):

```
%estructuras
estructura Persona:
    cadena nombre
    entero edad

%funciones
definir imprimirBienvenida():
    imprimir("Hola")
```

5.1.2 Comentarios

```
// Comentario de una sola línea
/* Comentario
   de varias líneas */
```

5.1.3 Tipos de datos

Tipo	Descripción	Ejemplo
entero	Número entero	42
flotante	Número de punto flotante	9.81
cadena	Cadena de caracteres	"hola"

caracter	Carácter único	'a'
bool	Valor booleano	verdadero o falso

5.1.4 Declaración de variables

```
entero edad = 20
flotante gravedad = 9.81
cadena nombre = "Comandante"
caracter inicial = 'a'
bool activo = verdadero
entero contador // sin valor inicial
```

5.1.5 Arreglos

```
entero numeros[5] = {1, 2, 3, 4, 5}
entero vacio[5] // sin valores iniciales
numeros[0] = 99
imprimir(numeros[1])
```

5.1.6 Estructuras

Los valores para instanciar una estructura se dan entre llaves y EN EL ORDEN en que se declararon los campos:

```
%estructuras
estructura Punto:
    entero x
    entero y

%funciones
definir main():
    Punto p1 = {10, 20}
    imprimir(p1.x)
```

5.1.7 Operadores

Categoría	Operadores	Notas
Aritméticos	+, -, *, /	El + también concatena cadenas
Incremento	++, --	Prefijo y sufijo
Relacionales	<, >	Producen un valor booleano
Igualdad	==, !=	Producen un valor booleano
Lógicos	&&,	Exigen operandos booleanos
Negación	!	Solo se aplica sobre booleanos

5.1.8 Condicionales

```
si (edad >= 18) entonces
    imprimir("Mayor de edad")
sino (edad == 18) entonces
    imprimir("Justo 18")
contrario
    imprimir("Menor de edad")

elegir (opcion):
    caso 1:
        imprimir("uno")
    romper
```

```
siempre:
    imprimir("otro")
```

5.1.9 Ciclos

```
mientras (contador < 10) hacer:
    contador = contador + 1

hacer:
    contador = contador + 1
mientras (contador < 10)

para (entero i = 0; i < 10; i++):
    imprimir(i)
```

5.1.10 Funciones

```
definir saludar(cadena nombre):
    imprimir("Hola")

definir calcular(entero fuerza) -> entero:
    entero total = fuerza * 2
    retornar total
```

5.1.11 Entrada y salida

```
imprimir("Hola mundo")
cadena nombre = leer()
```

5.2 Lenguaje Zetariano

Zetariano es un lenguaje orientado a objetos, de sintaxis parecida a Java. Cada archivo define UNA clase, y el archivo debe llamarse igual que esa clase. Es sensible a mayúsculas y minúsculas. Su extensión de archivo es .z.

5.2.1 Estructura de un programa

```
public class Persona {
    String nombre;
    int edad;

    public Persona(String nombreParametro, int edadParametro) {
        nombre = nombreParametro;
        edad = edadParametro;
    }

    public void saludar() {
        println("Hola, me llamo " + nombre);
    }
}
```

5.2.2 Comentarios

```
// Comentario de una sola linea
/* Comentario
de varias lineas */
```

5.2.3 Tipos de datos

Tipo	Descripción	Ejemplo
int	Número entero	42
double	Número de punto flotante	9.81
String	Cadena de caracteres	"hola"
char	Carácter único	'a'
boolean	Valor booleano	true o false

5.2.4 Atributos, constructor y métodos

Los atributos no llevan modificador de acceso obligatorio (aunque se acepta public o private, sin que afecte el comportamiento del programa). El constructor se puede sobrecargar, igual que los métodos:

```
public class Contador {
    int valor = 0;

    public Contador() { }

    public Contador(int inicial) {
        valor = inicial;
    }

    public void sumar(int cantidad) {
        valor = valor + cantidad;
    }

    public int obtenerValor() {
        return valor;
    }
}
```

Dentro de un método se puede llamar a otro método de la misma clase con o sin this. explícito: obtenerValor() y this.obtenerValor() son equivalentes.

5.2.5 Objetos y arreglos

```
Persona p1 = new Persona("Ana", 25);
p1.saludar();

int[] numeros = new int[5];
numeros[0] = 10;
int[] literal = {1, 2, 3};
```

5.2.6 Operadores

Categoría	Operadores	Notas
Aritméticos	+, -, *, /, %	El + también concatena cadenas
Incremento	++, --	Prefijo y sufijo
Asignación abreviada	+=, -=, *=	
Relacionales	<, >, <=, >=	Producen un valor booleano
Igualdad	==, !=	Producen un valor booleano (también contra null)
Lógicos	&&,	Exigen operandos booleanos
Negación	!	Solo se aplica sobre booleanos
Ternario	? :	int max = (a > b) ? a : b;

5.2.7 Condicionales

```
if (edad >= 18) {
    println("Mayor de edad");
} else if (edad == 18) {
    println("Justo 18");
} else {
    println("Menor de edad");
}

switch (opcion) {
    case 1:
        println("uno");
        break;
    default:
        println("otro");
}
```

5.2.8 Ciclos

```
while (contador < 10) {
    contador++;
}

do {
    contador++;
} while (contador < 10);

for (int i = 0; i < 10; i++) {
    println(i);
}
```

5.2.9 Entrada y salida

```
println("Hola mundo");    // con salto de linea
print("sin salto");       // sin salto de linea
String texto = readln();
```

5.3 Lenguaje Pig Latin

Pig Latin es el lenguaje ORQUESTADOR: todo programa completo empieza en un archivo .pig, que importa las clases (.z) y estructuras/funciones (.y) que necesite. Pig Latin en sí mismo no define clases, estructuras ni funciones propias —siempre las importa. Es sensible a mayúsculas y minúsculas.

5.3.1 Estructura de un programa

Un archivo .pig tiene tres secciones: los import (opcional), la sección VARIABLES (opcional) y la sección MAIOR (obligatoria, es donde corre el programa):

```
import Nodo.z
import Pila.z
import utils.utils.y

VARIABLES>
esto pila : novus Pila();
esto contador : numerus 0;

MAIOR>
```

```
>> "Hola mundo";  
FINIS
```

Un import de un archivo dentro de una subcarpeta se escribe con puntos: import utils.utils.y equivale al archivo utils/utils.y, relativo a la carpeta del propio .pig.

El cierre FINIS (en mayúsculas) del final del programa es opcional. La palabra finis en minúsculas, en cambio, se usa para cerrar bloques de control (condicionales y ciclos) y sí es obligatoria en esos casos.

5.3.2 Comentarios

```
// Comentario de una sola linea  
## Comentario  
de varias lineas ##
```

5.3.3 Tipos de datos

Tipo	Descripción	Ejemplo
numerus	Número entero	42
decimalis	Número de punto flotante	9.81
textum	Cadena de caracteres	"hola"
littera	Carácter único	'a'
bool	Valor booleano	verum o falsus

5.3.4 Declaración de variables

```
esto edad : numerus 20;  
esto gravedad : decimalis 9.81;  
esto nombre : textum "Comandante";  
  
## Forma abreviada para booleanos ##  
esto activo : verum;  
  
## Sin valor inicial ##  
esto contador : numerus;  
  
## Instanciar un objeto de una clase importada ##  
esto pila : novus Pila();  
  
## Instanciar una estructura importada ##  
esto p : Punto { x: 1, y: 2 };
```

5.3.5 Arreglos

```
series numeros[3] : numerus {1, 2, 3};  
series vacio[5] : numerus;  
numeros[0] = 99;  
>> numeros[1];
```

5.3.6 Objetos: campos y métodos

```
pila.apilar(5);  
>> pila.toString();  
esto tope : numerus pila.obtenerCima();
```

5.3.7 Operadores

Categoría	Operadores	Notas
Aritméticos	+, -, *, /	El + también concatena cadenas
Incremento	++, --	Solo como sufijo: contador++;
Relacionales	<, >, <=, >=	Producen un valor booleano
Igualdad	==, !=	Producen un valor booleano
Lógicos	&&,	Exigen operandos booleanos
Negación	non	Solo se aplica sobre booleanos

5.3.8 Condicionales

```
si (edad >= 18) {
  >> "Mayor de edad";
} aliter (edad == 18) {
  >> "Justo 18";
} aliter {
  >> "Menor de edad";
} finis;
```

5.3.9 Ciclos

```
dum (contador < 10) {
  contador++;
} finis;

facere {
  contador++;
} dum (contador < 10);

per (esto i : numerus 0; i < 10; i++) {
  >> i;
} finis;
```

perge (continuar con la siguiente vuelta) e interrompe (cortar el ciclo) solo se pueden usar dentro de un dum, facere o per. reddere retorna del programa principal.

5.3.10 Entrada y salida

```
>> "Hola mundo";
>> "Nombre: " >> nombre;
nombre <<;
<<; // lee y descarta el valor
```

6. Ejemplo Completo de Uso

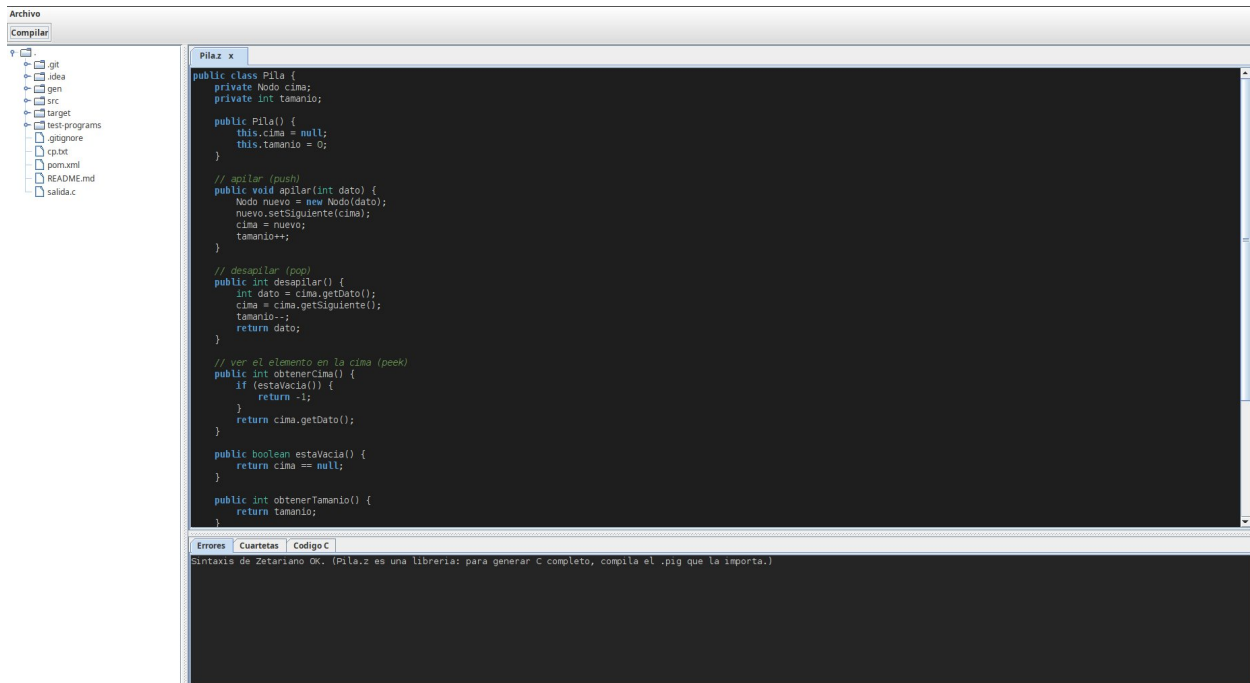
A continuación se describe un recorrido completo por la aplicación, usando el programa de ejemplo incluido en la carpeta test-programs/pila: una pila (estructura de datos) implementada con una clase Zetariano (Pila.z, que a su vez usa Nodo.z), un archivo de utilidades en Y? (utils/utils.y), y un programa principal en Pig Latin (main.pig) con un menú interactivo.

Paso 1: Abrir el programa

Ejecutar la aplicación (sección 2.3). Se abre la ventana principal.

Paso 2: Abrir la carpeta del proyecto

Ir a Archivo > Abrir carpeta y seleccionar la carpeta test-programs/pila. El árbol de archivos muestra la subcarpeta utils/ (con utils.y dentro) y los archivos main.pig, Nodo.z y Pila.z.



Paso 3: Revisar el programa principal

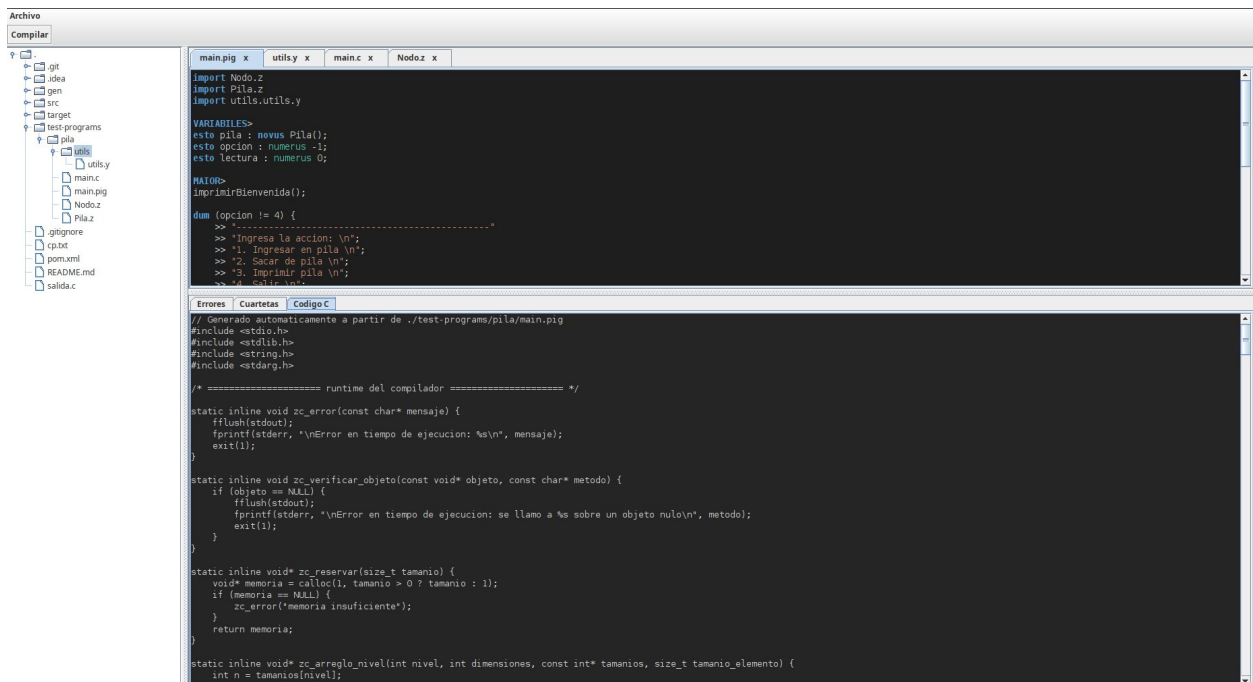
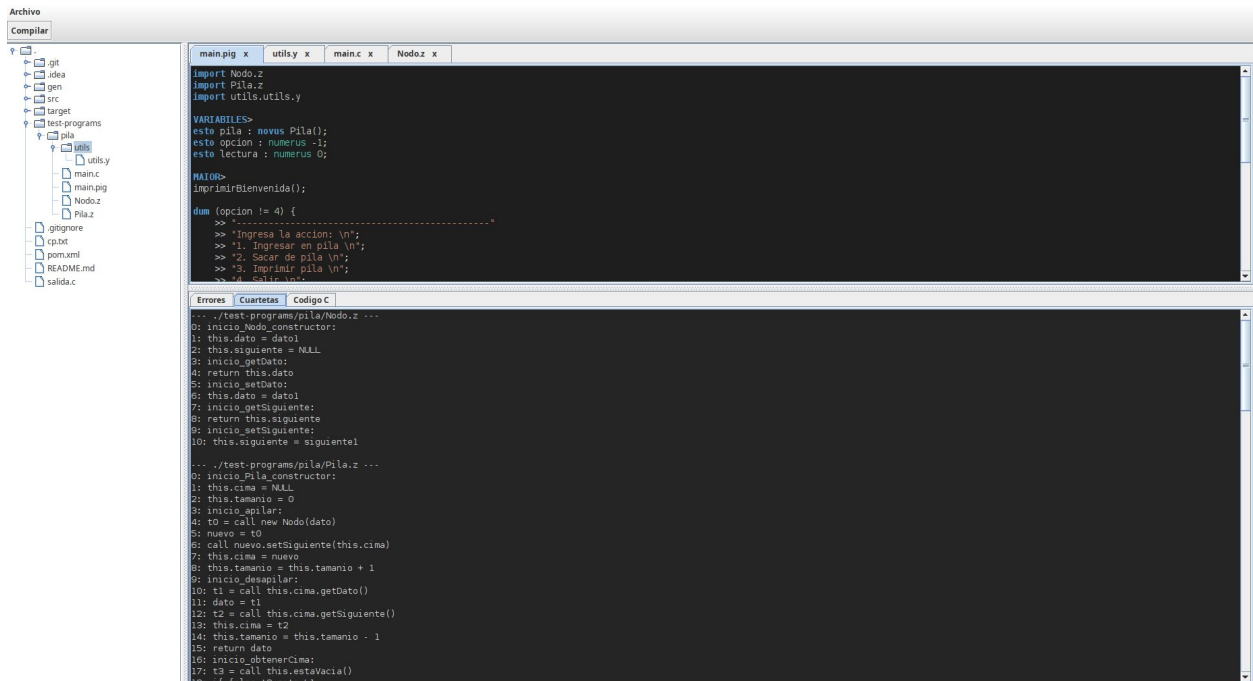
Abrir main.pig con doble clic. Al inicio del archivo se ven los tres import (Nodo.z, Pila.z y utils.utils.y); luego la sección VARIABLES declara el objeto pila como una instancia nueva de Pila; y la sección MAIOR contiene un ciclo con un menú de cuatro opciones: ingresar un número a la pila, sacarlo, imprimir la pila completa, o salir.

Paso 4: Compilar

Con la pestaña de main.pig activa, presionar el botón Compilar. La aplicación guarda automáticamente cualquier cambio pendiente en Nodo.z o Pila.z, compila los cuatro archivos en conjunto, y muestra el resultado.

Paso 5: Revisar los resultados

La pestaña Errores muestra “Compilación OK, sin errores”. La pestaña Cuartetas muestra el código de tres direcciones generado para cada uno de los cuatro archivos. La pestaña Código C muestra el programa en C completo, con una función por cada método de Pila y Nodo, por la función de utils.y, y el main correspondiente a la sección MAIOR de main.pig.



Paso 6: Ejecutar el programa

Compilar generó el archivo main.c junto a main.pig, dentro de la misma carpeta test-programs/pila. Desde una terminal, posicionada en esa carpeta:

```
gcc -std=c11 main.c -o main
./main
```

El programa saluda y muestra el menú de cuatro opciones. Siguiendo esta secuencia:

- Opción 1, valor 5 — se ingresa el número 5 a la pila.
- Opción 1, valor 7 — se ingresa el número 7 a la pila.
- Opción 3 — se imprime la pila completa.
- Opción 2 — se saca un elemento.
- Opción 4 — se sale del programa.

La salida obtenida, verificada de forma real, es la siguiente:

```
Pila [Cima -> Base]: 7, 5
Elemento desapilado: 7
Fin del programa
```

Dos comportamientos adicionales, también verificados: si en el menú se escribe algo que no es un número donde se espera uno, el programa lo toma como 0 y continúa (no se queda esperando una entrada válida para siempre); y si se elige la opción 2 (sacar) cuando la pila está vacía, el programa termina mostrando “Error en tiempo de ejecución: se llamó a Nodo.getDato sobre un objeto nulo” en vez de fallar de forma abrupta —porque Pila.z, tal como está escrito, no revisa si la pila está vacía antes de sacar un elemento.

7. Mensajes de Error y Solución

Los errores se clasifican en tres tipos, igual en los tres lenguajes: léxicos, cuando aparece un carácter que no pertenece al lenguaje; sintácticos, cuando la estructura del código no corresponde a ninguna forma válida; y semánticos, cuando la estructura es correcta pero el significado no lo es (una variable inexistente, un tipo incompatible, etc.).

Los mensajes semánticos aparecen en español, con el nombre exacto de la variable, el tipo o el método involucrado. Los mensajes léxicos y sintácticos, en cambio, provienen directamente de ANTLR y aparecen en un formato más técnico (por ejemplo extraneous input '(' expecting ')'); igual indican la línea y la columna exactas, así que conviene ir directo a esa posición del archivo indicado.

Mensaje (semántico)	Causa y solución
La variable 'x' no ha sido declarada	Se usa un nombre que nunca se declaró, o está mal escrito. Verificar la ortografía.
'x' ya fue declarada en este ámbito	Existen dos declaraciones con el mismo nombre en el mismo bloque o función. Cambiar uno de los nombres.
El operador 'X' no se puede aplicar entre A y B	La operación no es válida para esos tipos. Revisar la tabla de compatibilidad de tipos de la sección correspondiente del manual técnico.
La condición debe ser de tipo booleano, no X	La condición de un si/if/dum debe evaluar a verdadero/falso. Usar una comparación en vez de un número directamente.
No se puede asignar un valor de tipo X a una variable de tipo Y	La conversión implícita solo opera hacia arriba en la jerarquía de tipos. No se puede guardar, por ejemplo, un decimal en un entero.
No existe el método 'x' con N argumento(s) en la clase 'Y'	El método no existe, o se está llamando con una cantidad de argumentos distinta a la declarada.
'break'/'romper' solo se puede usar dentro de un ciclo o un switch/elegir	Se usó fuera de su contexto válido.
No es un arreglo, no se puede indexar	Se usó corchetes [] sobre una variable que no es un arreglo.
Falta inicializar el atributo / La	Al instanciar una estructura o clase con {...}, deben darse valores a

estructura tiene N campo(s) pero se le dan M valor(es)	todos sus atributos.
--	----------------------

8. Archivos Soportados

Extensión	Uso
.y	Código fuente en el lenguaje Y? (estructuras y funciones)
.z	Código fuente en el lenguaje Zetariano (una clase por archivo)
.pig	Código fuente en el lenguaje Pig Latin. Es el punto de entrada de un programa completo, y el único que se puede compilar a C directamente
.c	Código en C generado automáticamente al compilar un .pig. Se guarda junto al .pig con el mismo nombre

9. Notas Adicionales

- A diferencia de un simple analizador, el programa que resulta de compilar un .pig SÍ se ejecuta — pero el paso de ejecutarlo ocurre fuera de la aplicación, compilando el .c generado con un compilador de C y corriendo el ejecutable resultante (sección 4.11).
- Solo un archivo .pig se puede compilar de forma completa (genera cuartetos y código C). Un .z o un .y solo se puede verificar léxica y sintácticamente por separado, porque su significado completo depende del .pig que los importe.
- Antes de compilar, la aplicación guarda automáticamente todos los archivos abiertos con cambios pendientes, no solo el activo.
- El árbol de archivos se actualiza solo después de compilar, así que un archivo .c recién generado aparece sin necesidad de refrescar manualmente.
- Los tres lenguajes distinguen mayúsculas de minúsculas, tanto en las palabras reservadas como en los identificadores.
- Se pueden tener varios archivos abiertos al mismo tiempo en pestañas distintas; cada pestaña recuerda su propia ruta y muestra un asterisco (*) mientras tenga cambios sin guardar.